Università degli Studi di Napoli Federico II
Dipartimento di Ingegneria Elettrica e delle Tecnologie dell'Informazione
Corso di Laurea in Informatica

Laboratorio di Sistemi Operativi

Anno Accademico 2025–2026

Elaborato di gruppo – Traccia A

Labirinto con oggetti e uscita

Candidati

Molinari Rocco - N86005071

Megna Daniele - N86005120

Docenti

Prof. Alberto Finzi

Napoli, 19 febbraio 2026

Indice

1 Introduzione **2**
1.1 Descrizione del problema 2

2 Guida d'uso per il server e il client **3**
2.1 Descrizione tecnica e guida d'uso 3
2.1.1 Descrizione tecnica del sistema 3
2.1.2 Server 3
2.1.3 Guida d'uso del server 3
2.1.4 Client 4
2.1.5 Guida d'uso del client 4
2.1.6 Procedura completa di utilizzo 5

3 Comunicazione tra client e server **6**
3.0.1 Protocollo applicativo 6
3.0.2 Messaggi del protocollo 6
3.0.3 Flusso di comunicazione 7

4 Dettagli implementativi **8**
4.1 Generazione della mappa 8
4.2 Funzione shuffle 9

1 Introduzione

Il presente elaborato ha l'obiettivo di documentare la progettazione e lo sviluppo di un sistema distribuito client–server, realizzato in linguaggio C su piattaforma UNIX/Linux, che consente a più utenti di partecipare contemporaneamente a un gioco di esplorazione di un labirinto. Il progetto è stato realizzato dagli studenti Molinari Rocco e Megna Daniele, ed è stato sviluppato a fini valutativi nell'ambito dell'insegnamento di Laboratorio di Sistemi Operativi del Corso di Laurea in Informatica presso l'Università degli Studi di Napoli "Federico II".

1.1 Descrizione del problema

Saranno presentati la progettazione e lo sviluppo di un sistema distribuito client–server finalizzato all'implementazione di un gioco multiutente di esplorazione di labirinti. Il sistema consente la partecipazione simultanea di più giocatori, la gestione delle loro interazioni all'interno di una mappa bidimensionale generata dinamicamente e la determinazione dei risultati della partita sulla base degli obiettivi raggiunti. L'applicazione permette agli utenti di connettersi al server, esplorare il labirinto, raccogliere oggetti e tentare di raggiungere un'uscita, mentre il server mantiene il controllo dello stato globale del gioco e fornisce a ciascun client una vista parziale e personalizzata dell'ambiente. Il sistema gestisce inoltre la sincronizzazione delle informazioni, l'invio periodico di aggiornamenti e la registrazione delle attività principali, garantendo coerenza, scalabilità ed efficienza operativa.

2 Guida d'uso per il server e il client

2.1 Descrizione tecnica e guida d'uso

2.1.1 Descrizione tecnica del sistema

Il sistema implementa un'architettura distribuita di tipo client-server basata su comunicazione TCP. L'applicazione è realizzata in linguaggio C e utilizza le librerie POSIX per la gestione dei socket e dei thread.

2.1.2 Server

Il server è un'applicazione multithread:

- un thread principale accetta le connessioni in ingresso;
- per ogni client viene creato un thread dedicato;
- mutex e variabili di condizione garantiscono la sincronizzazione delle risorse condivise (mappa, punteggi, lobby).

2.1.3 Guida d'uso del server

Compilazione

```
gcc server.c map.c -o server -pthread
```

Avvio

Il server deve essere avviato per primo:

```
./server
```

All'avvio il server:

- inizializza i file necessari;
- genera la mappa di gioco;

- si mette in ascolto sulla porta 8080;
- attende le connessioni dei client.

Il server rimane in esecuzione fino a interruzione manuale.

2.1.4 Client

Il client utilizza due thread:

- uno per inviare i comandi dell'utente;
- uno per ricevere messaggi asincroni dal server.

La persistenza dei dati è garantita tramite file di testo: users.txt, score.txt, filelog.txt.

2.1.5 Guida d'uso del client

Compilazione

```
gcc client.c map.c -o client -pthread
```

Connessione al server

Il client si avvia specificando l'indirizzo IP del server:

```
./client <indirizzo_server>
```

Autenticazione

All'avvio l'utente può:

- registrare un nuovo account;
- effettuare il login con credenziali esistenti.

Le credenziali vengono salvate dal server nel file users.txt.

Comandi disponibili

Durante la partita il client può utilizzare i seguenti comandi:

- `W` – movimento verso l'alto;
- `S` – movimento verso il basso;
- `A` – movimento verso sinistra;

- `D` – movimento verso destra;
- `list` – visualizza i giocatori connessi;
- `exit` – abbandona la partita.

2.1.6 Procedura completa di utilizzo

1. Compilare server e client.
2. Avviare il server.
3. Avviare uno o più client.
4. Effettuare registrazione o login.
5. Attendere l'avvio della partita.
6. Muoversi nella mappa e raccogliere oggetti.
7. Attendere la fine della partita e la comunicazione del vincitore.

La partita termina allo scadere del tempo o quando un giocatore raggiunge l'uscita. Il server calcola il punteggio e determina il vincitore.

3 Comunicazione tra client e server

3.0.1 Protocollo applicativo

La comunicazione tra client e server avviene tramite un protocollo applicativo, basato sullo scambio di messaggi testuali. Il protocollo permette ai client di inviare richieste al server e ricevere aggiornamenti sullo stato del gioco. Il server gestisce la logica della partita e mantiene lo stato del labirinto e dei giocatori connessi. Il client invia comandi relativi alle azioni dell'utente (movimenti, richiesta lista giocatori, uscita dal gioco) e riceve dal server aggiornamenti della mappa o notifiche relative allo stato della partita. I messaggi del protocollo sono identificati tramite brevi stringhe o singoli caratteri che rappresentano i diversi tipi di operazione.

3.0.2 Messaggi del protocollo

Nella seguente tabella sono riportati i principali messaggi scambiati tra client e server.

Messaggio	**Direzione**	**Descrizione**
R	Client → Server	Richiesta di registrazione di un nuovo utente
L	Client → Server	Richiesta di login
Y	Server → Client	Operazione completata con successo
N	Server → Client	Operazione fallita
W	Client → Server	Movimento verso l'alto
A	Client → Server	Movimento verso sinistra
S	Client → Server	Movimento verso il basso
D	Client → Server	Movimento verso destra
list	Client → Server	Richiesta della lista dei giocatori connessi
B	Server → Client	Invio di una mappa aggiornata con effetto nebbia
M	Server → Client	Notifica che un giocatore ha raggiunto l'uscita
E	Server → Client	Fine della sessione di gioco
W	Server → Client	Vittoria del giocatore
L	Server → Client	Sconfitta del giocatore

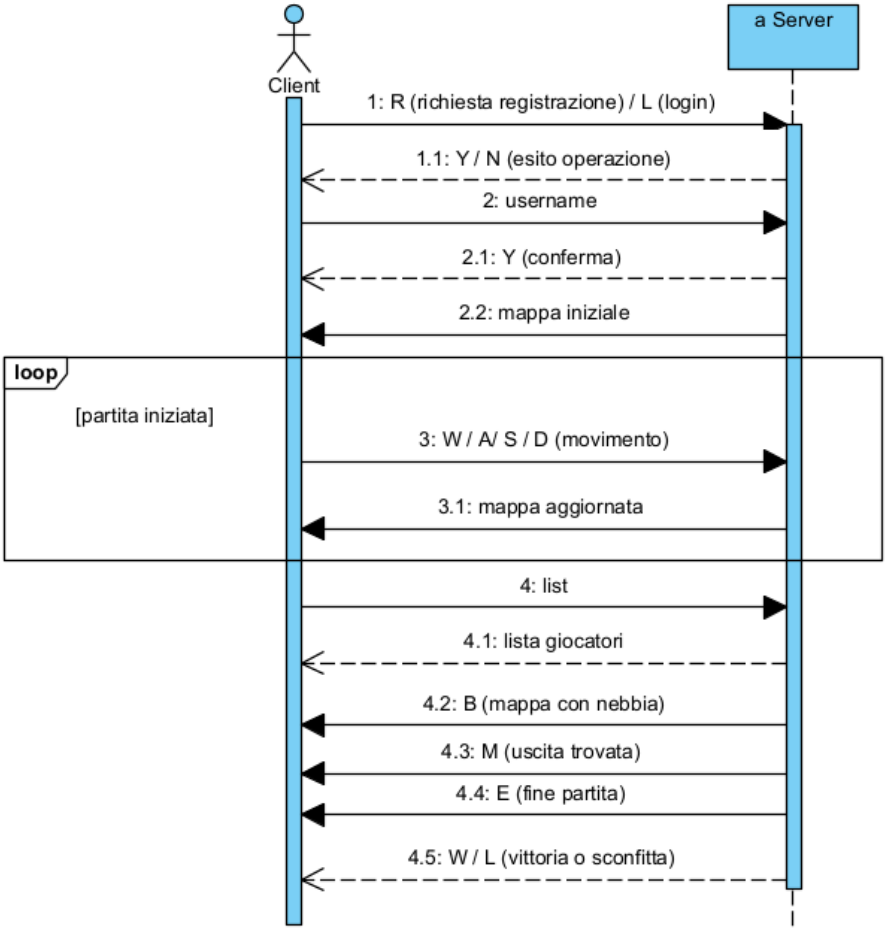
Tabella 3.1: Messaggi del protocollo applicativo

3.0.3 Flusso di comunicazione

Il funzionamento del protocollo segue una sequenza di interazioni tra client e server. Inizialmente il client effettua l'autenticazione tramite registrazione o login. Una volta autenticato, il client riceve la mappa iniziale del labirinto e può iniziare a inviare comandi di gioco. Durante la partita il client può inviare comandi di movimento oppure richiedere informazioni sui giocatori connessi. Il server elabora le richieste ricevute, aggiorna lo stato del gioco e invia ai client le informazioni aggiornate. Il ciclo di interazione continua finché la partita non termina. Nel codice del client il ciclo principale è implementato tramite un loop che continua finché la variabile di stato end non viene impostata a 1 dal thread che riceve notifiche dal server. Quando viene raggiunta la condizione di fine partita il server invia un messaggio di terminazione ai client connessi, indicando il risultato finale della sessione di gioco.

Sequence diagram

Il protocollo a livello di applicazione tra client e server lo illustriamo nel segente sequance diagram.



4 Dettagli implementativi

Come dettagli implementativi abbiamo pensato di parlare di queste due funzioni utilizzate:

- Generazione della mappa
- Funzione shufffle

4.1 Generazione della mappa

La nostra funzione, generatemap è una parte fondamentale del nostro progetto, perchè permette di generare una mappa del labirinto robusta e molto efficinete, abbiamo utilizzato un algoritmo di DFS(Depth-First Serach). Per prima cosa abbiamo scelto una dimensione della mappa sempre dispari in altezza e larghezza, perchè questo permette una chiara distinzione tra celle destinati ai muri e celle dedicate ai corridoi, in modo da evitare passaggi irregolari. Abbiamo utlizzato una allocazione dinamica in modo che ad ogni esecuzione verrà generata una mappa diversa, inoltre gestiamo anche possibili errori in memoria in modo tale da rendere stabile il programma. La mappa parte da uno stato di non visitato da parte dell'utente, e questo partirà dal centro della mappa. Abbiamo utilizzato l'inserimento automatico delle uscite per evitare casi in cui il labirinto venga generato senza uscite, infine abbiamo utilizzato una matrice ausiliaria visited che contiene le celle già esplorate dall'algoritmo DFS, questo permette di evitare cicli e duplicazioni durante la generazione del labirinto.

```
char **generateMap(int *width, int *height) {
 srand(time(NULL));

 // forza width e height dispari per evitare muri frammentati
 int w = (MINWIDTHMAP + rand() % (MAXWIDTHMAP - MINWIDTHMAP + 1)) | 1;
 int h = (MINHEIGHTMAP + rand() % (MAXHEIGHTMAP - MINHEIGHTMAP + 1)) | 1;

 *width = w;
 *height = h;

 char **map = malloc(h * sizeof(char*));
```

```
 int **visited = malloc(h * sizeof(int*));
 if(!map || !visited) return NULL;

 for(int i=0;i<h;i++) {
    map[i] = malloc(w * sizeof(char));
    visited[i] = malloc(w * sizeof(int));
    if(!map[i] || !visited[i]) {
        for(int j=0;j<i;j++) { free(map[j]); free(visited[j]); }
        free(map); free(visited);
        return NULL;
    }
    for(int j=0;j<w;j++) {
        map[i][j] = WALL;   // usa la costante WALL
        visited[i][j] = 0;
    }
 }
 // PARTENZA DAL CENTRO (pari/dispari coerente)
 int startX = h/2 | 1;
 int startY = w/2 | 1;

 dfs(map, visited, startX, startY, w, h);
 addExits(map, w, h);

 // libera visited
 for(int i=0;i<h;i++) free(visited[i]);
 free(visited);

 return map;
}
```

4.2 Funzione shuffle

La funzione shuffle permette di aumentare la varietà dei labirinti generati, questa funzione è basata sull'algoritmo di Fisher–Yates. Ciò garantisce percorsi diversi ad ogni esecuzione, dando al labirinto una struttura più naturale e imprevedibile.

```
void shuffle(int *arr, int n) {
 for (int i = n-1; i > 0; i--) {
     int j = rand() % (i+1);
```

```
        int tmp = arr[i];
        arr[i] = arr[j];
        arr[j] = tmp;
    }
}
```